A1-Component-Problem Statement Identification

Division - B

Team Members:

1) **Shlok Jaiswal - 202301040182**
2) **Mrugmai Dudhamande - 202301040190**
3) **Snehal Chavan - 202402040033**
4) **Pournima Khamble - 202402040034**
5) **Snehal Mane – 202301040076**

OpenHour

Real-Time Human Availability Marketplace

Problem Statement Document

1. Problem Statement — Relevance & Clarity

1.1 Statement

OpenHour is a real-time marketplace where individuals publish blocks of their available time, along with the date, location, relevant skill, and rate so that other users can discover and instantly claim that time for a specific need. Unlike traditional booking platforms where a request is sent and the provider must respond, OpenHour inverts the flow: availability is published first, as live inventory, and the first person to accept a published slot secures it.

1.2 Real-World Problem Being Solved

Across freelancing, tutoring, local services, consulting, and even informal peer help (e.g. a senior helping a junior debug code, or a neighbour offering an hour of electrical repair), the same friction repeats itself:

- **Discovery is slow:** Seekers must search directories, message multiple people, and wait for replies before knowing if anyone is actually free.
- **Availability is invisible:** A provider's free time exists only in their own head or calenda, it is not a discoverable, requestable resource.
- **Confirmation is uncertain:** Even after a request is sent, there is no guarantee of a timely response, leading to double-booking, no-shows, or wasted time on both sides.
- **Idle time is wasted economically:** A freelancer, tutor, or tradesperson with a free two-hour window has no low-friction way to monetize it on short notice.

OpenHour addresses this by treating time itself as a listable, searchable, and instantly claimable unit, similar to how ride-hailing apps turned "a free driver nearby" into a bookable resource in real time.

1.3 Relevance to the Domain

The problem sits squarely in the domain of full-stack web/software engineering and systems design, and is relevant because it requires hands-on application of:

- Client-server web application development (frontend for browsing/publishing, backend for matching and transactions)
- Database design and concurrency control (preventing two users from booking the same slot)
- API design and real-time communication (REST plus WebSockets for live slot status)
- Core CS fundamentals that map directly to interview and coursework topics: OOP design of entities (User, Slot, Booking), race-condition handling, and system architecture trade-offs

1.4 Scope

In Scope	Out of Scope (v1)
Publishing/browsing time slots (skill, date, location, rate)	Payment gateway integration (escrow logic only, not live payments)
Real-time first-come-first-served slot claiming	Native mobile apps (web-first MVP)
Search/filter by skill, location, price, time	AI-based provider recommendation/matching
Basic trust layer: ratings, verification flags	Insurance/legal liability framework for in-person meetups
Notifications for booking confirm/cancel	Multi-language/localization support

Scoping the MVP this way keeps the project achievable within an internship timeline while still exercising the full architecture end-to-end — publish, search, claim, confirm.

2. System Type, Context & Architecture

2.1 System Type

- **Transactional marketplace:** providers (supply) and seekers (demand) are matched around a bookable unit, the time slot.
- **Real-time system:** slot status (open/claimed/expired) must update live for every user viewing it, not just on page refresh.
- **Location- and time-aware system:** matching depends on geospatial proximity and temporal overlap, not just keyword search.

2.2 High-Level Architecture — Client-Server, Layered

The system follows a layered client-server architecture with a clear separation of concerns, which keeps the concurrency-sensitive booking logic isolated from the UI and easy to reason about:

Layer	Responsibility	Representative Tech
Client (Presentation)	Slot publishing form, search/browse UI, live map, booking confirmation screens	React / Next.js, WebSocket client
API / Application Layer	Request validation, auth, search endpoints, booking orchestration	Node.js/Express
Matching & Concurrency Engine	Atomic slot-claim operation; ensures exactly one acceptor per slot	Redis distributed lock / DB row-level locking
Real-Time Layer	Pushes live slot status changes to all connected clients	WebSockets / Socket.IO
Data Layer	Persists users, slots, bookings, ratings	PostgreSQL (relational, ACID) + Redis (cache/locks)
Notification Layer	Booking confirmations, cancellations, reminders	Push/SMS/email service (e.g. Firebase, Twilio)

2.3 Core Entities (Domain Model)

- **User:** id, name, role (provider/seeker/both), verification status, rating
- **Slot:** id, providerId, skill, date/time window, location (lat/long), rate, status (open/claimed/expired/completed)
- **Booking:** id, slotId, seekerId, status, timestamps, created only after a successful atomic claim
- **Rating:** bookingId, raterId, score, comment, created post-completion

2.4 Why This Architecture — Justification

- **Client-server, not peer-to-peer:** a central server is required to guarantee a single, consistent source of truth for slot status, critical since the entire premise ("first accepted request wins") depends on avoiding race conditions.
- **Layered separation:** isolating the matching/concurrency engine from the API layer means the booking-race logic can be unit-tested and reasoned about independently of UI or networking concerns.
- **Relational database for core data:** bookings and slots need ACID transactions, two users must never both be told they got the same slot, which favors PostgreSQL-style consistency over an eventually-consistent NoSQL store for this specific table.
- **WebSockets for live status:** polling would either feel slow (long intervals) or waste resources (short intervals); a push-based real-time layer is a natural fit for a system whose core value proposition is "live" availability.
- **Distributed lock (e.g. Redis SETNX or DB row lock):** the "first accepted request secures the slot" rule is fundamentally a race condition; an atomic claim operation is the only reliable way to guarantee exactly one winner under concurrent requests.

2.5 Deployment Context

Envisioned as a cloud-hosted web application (consistent with ongoing AWS Cloud Practitioner preparation): containerized API services behind a load balancer, a managed relational database (e.g. Amazon RDS), a managed cache/lock store (e.g. Amazon ElastiCache for Redis), and static frontend hosting via a CDN, allowing the system to scale horizontally as concurrent slot-claim traffic grows.

3. Active Participation & Collaboration

This section documents individual contribution during team discussion of the problem statement.

1. Proposed the core "publish-availability-first, first-accept-wins" concept and framed it as a marketplace inversion, distinct from typical request-and-wait booking apps.
2. Facilitated discussion comparing OpenHour to existing models (Uber, Calendly, Fiverr) to test whether the idea was genuinely differentiated or a repackaging of an existing solution.
1. Listened to and incorporated peer feedback on scope, e.g. narrowing the MVP to exclude payments and insurance to keep the project deliverable within the internship timeline.
2. Helped the team reach consensus on system boundaries by walking through concrete use cases (tutoring, local repair, freelance consulting) to stress-test whether the design generalized.

4. Critical Thinking & Justification

4.1 Assumptions Questioned

- **"Is instant-claim actually better than a request/approval flow?" -** Considered that a fully automatic first-come-first-served model removes provider control (they might not want to work with just anyone). Resolved by allowing providers to optionally toggle between auto-accept and manual-review modes, rather than forcing one behaviour.
- **"Will trust be strong enough without long user history?" -** Because bookings are fast and transactional, users don't get the slow trust-building time Airbnb or long-term freelance platforms allow. This justified prioritizing lightweight but immediate trust signals (ID verification, rating badges) over deep profile history.
- **"Does this need to be general-purpose or niche?" -** Weighed a broad "any skill" marketplace against a focused niche (e.g. only tutoring, or only local trade services). A general-purpose scope was chosen for the problem statement stage to demonstrate architectural generality, while flagging that a real launch would likely start niche to bootstrap trust and liquidity.

4.2 Validating Relevance

The problem was validated against three criteria before being finalized:

3. Does it map to a real, observable gap? Yes, existing platforms are either request-based (slow) or fixed-schedule (rigid), with no live-inventory model for human time.

4. Does it require non-trivial systems design, not just CRUD? Yes, the concurrency/race-condition problem at the core of "first accepted request wins" makes this a genuine systems design exercise, not a simple listing app.
5. Is it scoped to be buildable in the available timeframe? Yes, once payments, insurance, and AI-matching were explicitly deferred to a later phase.

4.3 Justification Summary

OpenHour was chosen over alternative ideas because it forces engagement with a concrete, well-known hard problem in distributed systems (atomic claim/race-condition handling) inside a project that is otherwise easy to explain and demo, making it well suited to demonstrate both architectural reasoning and practical implementation skill.